

Connecter une interface à la base BSO Portal

Référence pour les équipes qui veulent brancher leurs propres interfaces directement sur la base Supabase de Bwat Sekrèm : connexion, schéma des tables, valeurs autorisées, fonctions, triggers et règles de sécurité.

Projet : `cdfqltezhcssutyjtyjb`

PostgreSQL · Supabase

REST + Realtime auto

00 À lire avant de commencer

La base est **partagée** : plusieurs départements y écrivent. Trois règles d'or évitent de casser les données des autres.

① Ne jamais écrire les champs « auto-gérés »

Les soldes, montants calculés et codes générés sont posés par des **triggers**. Envoyés à la main, ils sont **ignorés ou recalculés**. Marqués **auto** dans ce guide.

② Bouger un solde = insérer une transaction, jamais le modifier à la main

On insère dans `transactions_epargne` / `transactions_credit`. Le trigger calcule et bloque les incohérences (retrait > solde, paiement > restant).

③ S'authentifier

Lecture et écriture passent par la RLS. Clé publique (anon) + connexion utilisateur (email/mot de passe) pour obtenir un jeton valide.

01 Se connecter à Supabase

Supabase expose automatiquement une **API REST** (et Realtime) au-dessus de PostgreSQL.

Deux façons : la librairie officielle, ou de simples appels HTTP.

Coordonnées du projet

ÉLÉMENT	VALEUR
URL	https://cdfqltezhcssutyjtyjb.supabase.co
REST	<URL>/rest/v1/<table>
Auth	<URL>/auth/v1
Clé anon	Clé publique (déjà dans l'app). Sûre à exposer : la RLS protège.

Avec @supabase/supabase-js

```
import { createClient } from '@supabase/supabase-js'
const supabase = createClient('https://cdfqltezhcssutyjtyjb.supabase.co', '<CLÉ_ANON>')

// 1) Se connecter
await supabase.auth.signInWithPassword({ email, password })
// 2) Lire (la RLS s'applique)
const { data } = await supabase.from('personnes').select('*').limit(20)
```

En HTTP brut

```
curl '.../rest/v1/comptes_epargne?select=*&limit=10' \
-H "apikey: <CLÉ_ANON>" -H "Authorization: Bearer <JWT>"
```

02 Le modèle de données

Tout tourne autour du **client**, qui possède des **comptes**, sur lesquels se passent des **transactions**. Un crédit s'adosse à un compte épargne (garantie).

```
personnes → comptes_epargne possède
personnes → comptes_credit possède
comptes_epargne → transactions_epargne reçoit
comptes_credit → transactions_credit reçoit
comptes_epargne → comptes_credit garantit
offres_plans → comptes_epargne définit le plan
profiles → personnes / transactions created_by
```

Relations principales (clés étrangères réelles). PK métier = UUID ; tables de référence = entiers.

03 Référence des tables métier

Champs **auto** = posés par trigger, ne pas envoyer. **requis** = obligatoire.

personnes **CLIENTS**

La fiche d'identité de chaque client.

COLONNE	TYPE	NOTE
id_personne	uuid	PK auto
prenom / nom	varchar	requis
code_client	text	auto
numero_telephone	text	contact
sexe	text	'M' / 'F'
piece_identification	bigint	FK
statut	text	Actif / Inactif
created_by	bigint	FK profiles.id

comptes_epargne **ÉPARGNE**

Un carnet d'épargne. Solde tenu par la base.

COLONNE	TYPE	NOTE
id_compte_epargne	uuid	PK auto
id_personne	uuid	FK
no_compte	text	auto
id_plan	integer	FK offres_plans
categorie_compte_epargne	text	Epargne / Fonds Garantie / Grandon
type_compte_epargne	text	Nouveau Compte / Upgrade / Staff / Bloque
solde_actuel	numeric	auto
succursale	bigint	FK

comptes_credit **CRÉDIT**

Un prêt. Capital final et restant calculés par la base.

COLONNE	TYPE	NOTE
id_compte_credit	uuid	PK auto
id_personne	uuid	FK
id_compte_epargne	uuid	FK garantie
type_compte_credit	text	contrainte DB (\$04)
montant_prete	numeric	requis
taux_interet	integer	% entier
duree_credit_mois	integer	requis
montant_final	numeric	auto
paiement_cumule / montant_restant	numeric	auto

transactions_epargne MOUVEMENTS

Chaque opération sur un carnet. À insérer pour bouger un solde.

COLONNE	TYPE	NOTE
id_transaction_epargne	uuid	PK
id_compte_epargne	uuid	FK requis
type_transaction	varchar	requis D/R/FL/S/V/FA
montant	numeric	requis
is_solde_initial	boolean	true = report d'ouverture
solde_avant / solde_apres_transactions	numeric	auto
virement_to / virement_from	text	n° compte (type V)

transactions_credit MOUVEMENTS

Chaque opération sur un prêt.

COLONNE	TYPE	NOTE
id_transaction_credit	uuid	PK
id_compte_credit	uuid	FK requis
type_transaction	varchar	requis Paiement / Penalite / Garantie
montant	numeric	requis
paiement_cumule / montant_restant	numeric	auto

Et toutes les autres ?

La base compte **22 tables**. Les 5 métier ci-dessus sont le cœur ; l'**annexe A** (fin de document) liste chacune des 22, par famille.

04 Valeurs autorisées (énumérations)

À respecter à l'écriture. Seul `type_compte_credit` est verrouillé par la base ; les autres par l'application.

CHAMP	VALEURS	VERROU
transactions_epargne.type_transaction	D dépôt · R retrait · FL nouveau livret · S frais service · V virement · FA frais auto	App
transactions_credit.type_transaction	Paiement · Penalite · Garantie	App+trigger
comptes_epargne.categorie_compte_epargne	Epargne · Fonds Garantie · Grandon	App
comptes_epargne.type_compte_epargne	Nouveau Compte · Compte Upgrade · Compte Staff · Compte Bloque	App
comptes_credit.type_compte_credit	Credit Cash · Konfyans · Electromenager · Lekol Ptit · Loyer	DB ✓
personnes.statut	Actif · Inactif	App
profiles.role	1 Admin · 2 Manager · 3 Agent · 4 Agent terrain	FK roles

05 Les fonctions de la base

Les fonctions de trigger (§06) tournent seules. Les fonctions ci-dessous s'appellent via `supabase.rpc('nom', { ... })`.

Sécurité (renvoient un booléen)

FONCTION	RÔLE
<code>has_access_to_client(text)</code>	Peut-on modifier ce client ?
<code>has_access_to_compte_epargne(uuid)</code>	... ce compte épargne ?
<code>has_access_to_compte_credit(uuid)</code>	... ce compte crédit ?
<code>has_access_to_transaction_epargne(uuid)</code>	... cette transaction épargne ?
<code>has_access_to_transaction_credit(uuid)</code>	... cette transaction crédit ?

Métier & utilitaires

FONCTION	RETOUR	USAGE
<code>generate_code_client_simple(prenom, nom)</code>	text	Fabrique un code client.
<code>rechercher_utilisateur(search_text)</code>	record	Recherche floue (pg_trgm/unaccent).
<code>get_complete_schema()</code>	jsonb	Renvoie tout le schéma en JSON.
<code>calcul_solde_total(id_personne)</code>	record	legacy signature entier ≠ PK UUID — vérifier.
<code>cloturer_credit(id_compte_credit)</code>	void	legacy tester avant usage.

Recherche floue

Extensions `pg_trgm`, `fuzzystrmatch`, `unaccent` installées : `similarity()`, `soundex()`, `levenshtein()`, `unaccent()`.

06 Les triggers : ce qui se passe tout seul

Ils s'exécutent à chaque écriture et calculent les soldes / bloquent les erreurs. Vous n'avez pas à le faire côté interface.

SUR	QUAND	FAIT QUOI
<code>transactions_epargne</code>	AVANT insert	Recalcule le solde (D ajoute, R/FL/S/FA/V retirent), bloque négatif.
<code>transactions_credit</code>	AVANT insert	Met à jour payé/restant, refuse paiement > restant.
<code>comptes_credit</code>	AVANT ins/màj	Calcule <code>montant_final</code> = capital + intérêts.
<code>personnes</code>	AVANT insert	Génère le code client ; anti-doublon.
<code>comptes_epargne</code>	AVANT insert	Génère le <code>no_compte</code> .
<code>tables métier</code>	APRÈS màj/suppr	Journalise (<code>agent_action_audit</code>) ; suppressions → <code>sync_deletions</code> .
<code>presque toutes</code>	AVANT màj	Met à jour <code>updated_at</code> (synchro).

Conséquence pratique

Pour un dépôt, envoyez seulement `id_compte_epargne`, `type_transaction='D'`, `montant`. Pas les soldes : le trigger s'en charge.

07 Sécurité : RLS & autorisations

Couche 1 – RLS (dans la base)

OPÉRATION	RÈGLE ACTUELLE
SELECT (lire)	autorisé (true)
INSERT (créer)	autorisé si connecté
UPDATE / DELETE	autorisé si connecté – auth.uid() IS NOT NULL

Décision d'architecture

L'écriture est ouverte à tout utilisateur connecté pour que la **synchronisation ne soit jamais bloquée**. Le contrôle fin se fait à la couche 2 ; chaque action reste tracée.

Couche 2 – Autorisations (grants)

- Un **admin** (role=1) peut tout modifier.
- Un agent a besoin d'un **grant temporaire** (`temporary_access_grants`) pour éditer une fiche d'un autre — accordé par un admin, avec expiration.
- Appliqué par l'app via `has_access_to_*`.

Important pour vos interfaces

La RLS n'impose plus le grant : **votre interface doit vérifier les droits** (via `has_access_to_*`) pour garder le même contrôle que l'app principale.

08 Recettes d'intégration

Lire les comptes d'un client

```
await supabase.from('comptes_epargne')
  .select('id_compte_epargne, no_compte, categorie_compte_epargne, solde_actuel')
  .eq('id_personne', clientId)
```

Enregistrer un dépôt (le trigger calcule le solde)

```
await supabase.from('transactions_epargne').insert({
  id_compte_epargne: compteId, type_transaction: 'D', montant: 500
  // PAS de solde : le trigger le pose
})
```

Enregistrer un remboursement

```
await supabase.from('transactions_credit').insert({
  id_compte_credit: creditId, type_transaction: 'Païement', montant: 250
}) // montant > restant ⇒ le trigger REFUSE
```

Astuce

Appelez `get_complete_schema()` (RPC) pour récupérer le schéma à jour en JSON — idéal pour générer formulaires/documentation.

09 Checklist par département

- Obtenir la clé anon + un compte utilisateur (rôle adapté) auprès de l'admin.
 - Se connecter (`signInWithPassword`) avant toute écriture.
 - Ne jamais écrire les champs `auto` (soldes, montants calculés, codes).
 - Bouger les soldes uniquement via des transactions.
 - Respecter les valeurs autorisées (\$04).
 - Vérifier les droits avec `has_access_to_*` avant de laisser modifier.
 - Gérer les erreurs de trigger (retrait > solde, paiement > restant).
-

A **Annexe : les 22 tables, une par une**

Inventaire complet par famille. « Lignes » = enregistrements réels à l'extraction (0 = table vide/non utilisée).

Cœur métier (5)

TABLE	RÔLE	COLONNES CLÉS
personnes	Fiches clients	id_personne, code_client, prenom, nom, statut
comptes_epargne	Carnets d'épargne	id_compte_epargne, id_plan, categorie, solde_actuel
comptes_credit	Prêts	montant_prete, montant_final, montant_restant
transactions_epargne	Mouvements épargne	type_transaction, montant, is_solde_initial
transactions_credit	Mouvements crédit	type_transaction, montant, montant_restant

Listes de référence (5)

TABLE	LIGNES	RÔLE · COLONNES
offres_plans	11	Plans d'épargne · id_plan, nom_plan, montant
roles	4	Libellés des rôles · id, nom_role
piece_identification	4	Types de pièce (NIF/CIN/Passeport) · id, name
Succursale	2	Agences · id, name, Adresse
categorie_compte	0	Catégories (vide) · id, name

Utilisateurs & accès (2)

TABLE	RÔLE	COLONNES CLÉS
profiles	Comptes utilisateurs + rôle	user_id, email, firstname, name, role
temporary_access_grants	Autorisations temporaires	agent_id, granted_by, expires_at, scope_type

Journaux & contrôle (4)

TABLE	LIGNES	RÔLE · COLONNES
agent_action_audit	311	Qui a modifié/supprimé quoi (avant/après) · actor_id, action, before_data, after_data
access_grant_audit	6	Historique octrois/retraits d'accès · grant_id, event_type
historique_irregularites	7	Écarts solde déclaré vs calculé · no_compte, ecart, statut_controle
historique_actions	0	Journal d'actions (ancien, vide)

Système & synchronisation (3)

TABLE	LIGNES	RÔLE · COLONNES
active_sessions	152	Sessions d'appareils · user_id, device_fingerprint, last_active
sync_deletions	–	Suppressions à propager · table_name, record_id
personnes_roles	0	Liaison personne↔rôle (ancien, vide)

Sauvegardes de migration (3) – ne pas utiliser

Tables techniques historiques

Photos de sécurité prises avant la correction du taux d'intérêt (fév. 2026). RLS désactivée, aucune utilité — à ignorer :

`comptes_credit_amounts_backup_20260211_fix_percent` , `comptes_credit_taux_interet_backup_20260211` ,
`comptes_credit_taux_interet_backup_20260211_int_percent` .

Total : 5 métier + 5 référence + 2 accès + 4 journaux + 3 système + 3 sauvegardes = **22 tables**.

Guide technique d'intégration — structure réelle extraite de la base BSO Portal (Supabase / PostgreSQL, projet cdfqltezhccssutyjyb). Les noms en monospace sont exacts. La clé de service et le token d'administration ne figurent pas ici : ne jamais les exposer.